Автономная некоммерческая организация высшего образования
«Университет Иннополис»

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(БАКАЛАВРСКАЯ РАБОТА)
по направлению подготовки
09.03.01 «Информатика и вычислительная техника»

FINAL QUALIFICATION WORK
(BACHELOR'S THESIS)
Academic Program
09.03.01 «Computer Science»

Направленность (профиль) образовательной программы
«Информатика и вычислительная техника»

Field of Study:
«Computer Science»

Тема	**Разработка системы мониторинга производительности и обнаружения регрессий в Android-приложениях**
Topic	**Development of a Performance Monitoring and Regression Detection System for Android Applications**

Работу выполнил / *Prepared by*	**Миннемуллин Наиль Фидаилевич / Nail Minnemullin**	подпись / signature
Руководитель выпускной квалификационной работы / *Final Qualification Work Supervisor*	**Маццара Мануэль / Manuel Mazzara**	подпись / signature

Иннополис, Innopolis, 2026

Contents

List of Tables

List of Figures

Abstract

Chapter 1

Introduction

1.1 Background and motivation

Mobile applications are now an essential part of everyday life, and users expect them to be fast, responsive, and stable. Poor performance (e.g. slow startup, high CPU usage, or freezing UI) can quickly degrade the user experience and result in negative reviews [1], [2]. Because of this, performance monitoring has become a crucial part of modern software development.

Performance analysis methods for mobile applications can be divided into two main categories: pre-release profiling and post-release monitoring. Pre-release profiling provides detailed insights under controlled laboratory conditions but cannot accurately represent the wide variety of real devices and user behaviours. This limitation is critical because of Android fragmentation - the large variety of devices, operating system versions, and hardware configurations. The same application may perform differently across devices, which makes lab-based testing results less reliable [3], [4]. In contrast, post-release monitoring reflects actual user conditions but often lacks automated alerting mechanisms. Existing

solutions either do not provide alerting at all or rely on very limited predefined thresholds, which require tuning and can easily lead to inaccurate detection. As a result, developers need a way to continuously monitor how an application behaves on real devices under real usage conditions, with the ability to detect performance regressions automatically.

1.2 Research Gap

Current research and existing tools focus on either pre-release profiling (e.g., multi-layer profiling, energy or storage analysis) [5]–[7] or post-release monitoring and tracing in the wild [8]–[11]. However, none of them integrate automated performance regression detection. Tools that do provide alerting, such as Firebase Performance Monitoring, rely on hardcoded thresholds, which can lead to a large number of false positives or false negatives [2]. At the same time, studies on regression detection are developed independently and are not connected with continuous monitoring of Android applications [12]–[14]. As a result, there is no existing solution that combines continuous post-release performance monitoring with server-side regression detection. This gap is what the current work aims to address.

1.3 Research Questions and Aim

The aim of this thesis is to develop a system for continuous collection of performance metrics from running Android application and detect performance regressions in them. The system will collect key metrics directly from users' devices, send data to a backend, and notify developers when performance regres-

sions are detected.

The study addresses the following research questions:

1. How to implement a low-overhead, continuous performance monitoring system for Android applications that can be deployed on real user devices without significantly impacting the user experience?

2. How to accurately detect performance regressions using statistical methods, while minimizing false alerts.

3. How effective is the proposed system at detecting performance regressions when tested on real Android applications?

1.4 Novelty and Contribution

The novelty of this work is in combining post-release monitoring with automated performance regression detection in a single tool. The proposed solution introduces a low-overhead, continuous monitoring system designed for Android's fragmented ecosystem. Instead of relying on static thresholds, the system compares current performance against a historical baseline using statistical methods, so developers do not need to configure thresholds manually. As a result, they can receive timely and more reliable notifications about performance regressions and fix problems before they affect a significant number of users.

1.5 Structure

The thesis is organized as follows.

Chapter 2 reviews previous research and related work on mobile application performance monitoring and performance regression detection. It discusses existing pre-release profiling and post-release monitoring approaches, summarizes the performance metrics used in earlier studies, and highlights the research gap that motivates this work.

Chapter 3 describes the design of the proposed system. It explains the system requirements, the high-level architecture, the design of the mobile SDK and the backend server, and the regression detection methods that were considered, including the approach selected for this work.

Chapter 4 presents the implementation of the system, covering the mobile SDK, the backend server, the frontend dashboard, and the performance regression detection pipeline.

Chapter 5 evaluates the proposed system. It describes the validation setup, measures the overhead introduced by the SDK, compares the accuracy of the collected metrics against reference tools, evaluates the regression detection on synthetic regressions, and discusses the results together with their limitations.

Finally, Chapter 6 concludes the thesis by summarizing the work, answering the research questions, stating the main contribution, and outlining directions for future work.

Chapter 2

Literature Review

This chapter provides an overview of existing research related to mobile applications profiling and monitoring, It examines the methods used to collect performance metrics and the various types of metrics that are considered important for mobile application development. In addition, it discusses different approaches to anomaly detection, providing a foundation for selecting the most suitable method for this study.

The review is divided into three main parts:

- Section 2.1 compares two main approaches to collect performance metrics in mobile applications: pre-release profiling and post-release monitoring. This section outlines their main ideas, methodologies, and trade-offs, showing the advantages and limitaitons of each approach.
- Section 2.2 focuses on the metrics used in previous research, summarizing which ones are most frequently applied and explaining how they contribute to the assessment of application performance and user experience.
- Section 2.3 provides an overview of existing anomaly detection approaches,

ranging from simple threshold-based methods to advanced statistical and machine learning algorithms.

Together, these sections show the current state of research, providing a foundation for identifying the gaps that the the proposed SDK aims to address.

2.1 Performance monitoring

Performance is a critical aspect in mobile application development, as it directly affects user experience and retention. Performance issues such as slow response times, high energy consumption, or UI lags can lead to poor user experience, which may ofter result in negative ratings and uninstalls. The ability to automatically detect such performance problems can significantly improve application quality and positively impact user satisfaction, as shown by Tang *et al.* [1]. Profiling and monitoring are the two main strategies used to understand and improve application performance. Profiling can be used during development or testing and provides deep insights through high-resolution data collection. Monitoring, in contrast, takes place after the application is released and runs on real user devices. It must introduce as little overhead as possible to prevent any interference with normal app usage.

The Android ecosystem adds complexity to performance evaluation due to fragmentation - the wide variety of hardware, operating system versions, and user behaviors. [4]. This diversity makes it difficult to reproduce rare or device-specific issues found in the wild. The same app may behave differently on various devices depending on their environment, OS version, and hardware specifications [3]. This problem motivates research into tools and methods for collecting and analyzing mobile performance data effectively.

2.2 Techniques for Performance Data Collection

2.2.1 Pre-release Profiling Techniques

Several tools have been developed to support detailed performance profiling during development. One example is ProfileDroid by Wei *et al.* [5], which uses a multi-layer approach to profiling. Their method involves gathering data from four layers: application metadata, user interface interaction traces, operating system metrics(such as CPU or memory usage), and network activity. This allows developers to observe how different components interact. For example, how a UI event may trigger a CPU spike or a network request. The main strength of ProfileDroid is its ability to correlate performance across layers, giving a complete system-level view. However, its high overhead and dependence on a controlled environment limit its practical use on real devices.

Another relevant project is AndroStep [6], which focuses on analyzing Android storage performance during I/O operations. The authors found that existing Android benchmarking tools did not simulate realistic app behavior, especially missing operations like fsync() followed by 4 KB random writes — a common and essential operation used by SQLite [6]. Their tool provides valuable insights for benchmarking Android storage, but it only coveres storage metrics and requires manual execution, making it useful only for testing environment.

PowDroid, introduced by Bouaffar *et al.* in 2021 [7], was designed to measure the energy consumption of applications. However, like most pre-release tools, it works only under controlled conditions, which makes it difficult to test applications on a large range of real devices.

In the field of UI performance, a new state-of-the-art work, GuiWatcher, was

proposed by Liu *et al.* [15]. They presented an advanced way to detect UI lags and freezes. GUIWatcher uses computer vision techniques on screen recordings and is able to detect three types of problematic frames: "junky" frames (dropped or delayed ones), long loading frames (lengthy transition), and frozen frames. This method reflects what users actually see rather than relying only on timing logs. However, it is computationally expensive and suitable only for use during development or continuous integration testing. It also raises privacy concerns since it requires recording the device screen.

Finally, ADEL by Zhang *et al.* [16] provides a method to track network performance and detect energy leaks in Android applications by extending the platform with taint-tracking to trace how downloaded data is used within apps. It identifies unnecessary network activity that increases energy consumption. However, the approach is not suitable for continuous monitoring, as it requires modifications to the Android system and introduces high overhead, making it impractical for deployment on real user devices.

2.2.2 Post-release Monitoring Techniques

The main idea of post-release profiling is to collect metrics from apps as they run on real user devices. The main advantage of this approach is that it captures realistic usage patterns and real-world diversity that are hard to reproduce in a lab. The main challenge is to minimize overhead to avoid affecting user experience.

One of the first studies in this area was AppInsight [8], which introduced a method for instrumenting applications without modifying their source code. The framework automatically injects instrumentation into the compiled bytecode and collects runtime traces. By collecting traces from 30 users over several months,

the authors were able to identify performance bottlenecks that were not visible during development. However, this approach is limited because developers cannot easily add custom metrics.

Another example is Hubble [9], which is currently integrated into all Huawei devices. The main feature of Hubble is that it monitors the performance in an efficient in-memory buffer, where older data is constantly overwritten [9]. The relevant metrics are only persited when a performance issue is detected, providing engineers with detailed runtime traces right before an anomaly occurs. This approach combines high efficiency with low storage cost.

PerfProbe [11] is another post-release performance profiling tool that provides cross-layer insights to identify the root causes of performance issues. It monitors both application and system layers on mobile devices, collecting runtime traces with low overhead. Using statistical analysis, PerfProbe detects critical functions that deviate from normal performance and correlates them with system-level resource factors such as CPU, memory, or I/O usage. This allows developers to understand not only where the slowdown occurs but also why. In experiments with 22 Android apps, PerfProbe successfully diagnosed real-world issues and helped developers to decrease the user-perceived latency of 6 applications by 32-86% [11]

Another important work is Panappticon [10], a lightweight tracing system that captures performance across application, system, and kernel layers. It automatically identifies critical execution paths by linking user inputs to display updates. Unlike AppInsight, Panappticon can detect problems caused by interactions between multiple apps or system components. A one-month study showed it had only about 6% performance overhead, proving it suitable for continuous real-world monitoring.

2.3 Performance Metrics

Performance metrics are the foundation of profiling and anomaly detection. Commonly used metrics include:

- **CPU usage**: High CPU utilization often leads to reduced responsiveness. Overloaded threads or inefficient scheduling can cause UI lag. Studies have shown CPU contention to be a frequent cause of poor user experience on Android [17].

- **Memory usage**: Memory consumption is critical for app stability and avoiding resource leaks. An app that uses excessive RAM can trigger the system's low-memory killer or frequent garbage collection pauses. Developers, therefore, profile heap usage to catch allocation bottlenecks and memory leaks [18]. Keeping memory usage in check helps prevent crashes and ensures smooth operation as the app scales in features or data.

- **I/O operations**: Storage I/O significantly affects latency. This is especially true on mobile devices with slower flash storage or SD cards. For example, opening a new screen can take twice as long as normal due to slow disk I/O [6].

- **Energy consumption**: Battery life is one of the most important factors for users, because they are likely to notice and uninstall apps that drain the battery excessively. In fact, surveys show battery life often tops performance concerns for smartphone users, and many will avoid apps they perceive as energy-greedy [19]. Therefore, profiling energy usage and detecting "energy leaks" is a key part of mobile optimization.

- **UI responsiveness**: This metric reflects how quickly the user interface responds to user actions. Even when CPU, memory, and other performance indicators appear normal, delayed or uneven UI updates can significantly reduce user satisfaction. Both research and industry standards emphasize the importance of minimizing user-perceived latency. For instance, according to Google's RAIL model, if a response takes longer than approximately one second, the user's attention is likely to shift elsewhere [11]. Therefore, smooth scrolling, quick startup, and minimal frame drops are essential for maintaining a positive user experience.

Many tools combine serveral metrics to provide a comprehensive view. For example, ProfileDroid and Panappticon correlate UI and system activity, while PerfProbe connect CPU and I/O data with execution paths. The choice of metrics depends on the study goal: detailed pre-release profiling or lightweight post-release monitoring.

2.4 Performance Regression Detection

A performance regression is a situation where software still functions correctly but performs worse than before — for example, slower response times or higher resource usage. In mobile systems, this can appear as increased frame rendering time, slower startup, higher CPU usage, or excessive memory consumption after an application update. The goal of regression detection is not to find individual outliers but to identify degradation (i.e performance regression) across a user population.

2.4.1 Regression Detection in Software Systems

Performance regression detection has been actively studied in the context of large-scale software systems. Nguyen *et al.* [20] proposed one of the first automated approaches, using statistical process control techniques called control charts. Their method builds control limits from a baseline test run and scores new runs against them. If the violation ratio exceeds a threshold, the run is flagged as having a regression. The approach was evaluated on both industrial and open-source systems and showed accurate detection with simple and interpretable results.

Shang *et al.* [21] extended this direction by using regression models built on clustered performance counters. Instead of analyzing individual counters, they grouped related counters into clusters and built models to detect regressions automatically, avoiding the need for manual selection of target metrics.

A recent large-scale example is FBDetect by Yoon *et al.* [22], Meta's in-production regression detection system presented at SOSP 2024. FBDetect monitors around 800,000 time series and can detect regressions as small as 0.005%. It achieves this by measuring performance at the subroutine level rather than the service level, which significantly reduces noise. FBDetect uses statistical hypothesis testing to validate detected regressions and filters out false positives caused by transient issues such as load spikes or rolling updates. This work demonstrates the importance of combining statistical validation with practical filtering techniques in production environments

2.4.2 Mobile-Specific Gaps and Considerations

While regression detection is well explored in cloud and enterprise systems, mobile environments add unique constraints such as hardware diversity, limited resources, and privacy concerns. Systems like Carat [23] demonstrate a crowd-based approach to mobile performance analysis. Carat collects energy usage data from many users and builds a baseline of normal behavior for each application, identifying outliers through statistical normalization across similar devices.

Schmidt *et al.* [24] proposed an architecture that combines lightweight on-device feature extraction with remote analysis on a server. Although their study was based on older Symbian devices, the core idea of moving complex analysis to the server side remains practical for mobile performance monitoring today.

However, despite the progress in both server-side regression detection and mobile monitoring, there is still no solution that combines continuous post-release performance monitoring on Android devices with automated server-side regression detection. Existing mobile monitoring tools either do not provide alerting at all or rely on hardcoded thresholds. At the same time, the statistical methods successfully used in large-scale systems (such as percentile comparison and hypothesis testing) have not been applied to continuous monitoring of mobile applications. This gap is what the proposed system aims to address.

2.5 Summary

This chapter reviewed existing research on mobile performance profiling, monitoring, and regression detection. Pre-release tools such as ProfileDroid, AndroStep and PowDroid offer deep system-level insights but introduce high

overhead and require controlled environments, making them unsuitable for real user devices. Post-release monitoring systems, including AppInsight, PerfProbe and Hubble, solve this problem by collecting performance data directly from real users with minimal overhead.

The chapter also covered key performance metrics used in mobile applications, such as CPU usage, memory consumption, I/O operations, energy usage, and UI responsiveness. Finally, it reviewed existing approaches to performance regression detection, from control charts to large-scale industrial systems like FBDetect, showing that statistical methods can effectively identify sustained performance degradation.

Despite this progress, there is still no solution that combines continuous, low-overhead post-release monitoring with automated server-side regression detection specifically designed for Android applications. This gap motivates the development of the proposed system.

Chapter 3

Methodology

This chapter describes the design of the proposed performance monitoring system. Section 3.1 lists the requirements that influenced the design choices. Section 3.2 gives a high-level overview of the architecture and shows how the components work together. Sections 3.3, 3.4 describe the details of the mobile SDK and the backend server. Section 3.4.2 discusses the regression detection methods that were considered and highlights the selected one.

3.1 System requirements and constraints

Since the SDK (Software Development Kit) runs on real user devices, it must be lightweight so that it doesnt affect the user experience of the host application. Moreover, the system must be robust across different Android devices. Finally, regression detection must be reliable and resilient to outliers, as production data is often noisy.

3.2 High-Level System Architecture

The system consists of four components: the mobile SDK (client), the backend server, the frontend, and the regression detection engine. Figure 3.1 shows how they are connected.

The mobile SDK is a library that developers integrate into their Android applications. It collects performance metrics on users' devices and sends them to the backend for further analysis.

The backend receives metrics from user devices, stores them efficiently, and provides REST APIs for the frontend. It is also responsible for user authentication and project management.

The frontend is a web application that lets developers manage projects, browse metrics, configure alerts, and review detected regressions. It interacts with the backend through REST APIs to retrieve data and present it in a user-friendly way.

The regression detection engine continuously analyses the incoming metrics to find signs of performance degradation. If such a regression is detected, it sends an alert to developers using the configured notification channels.

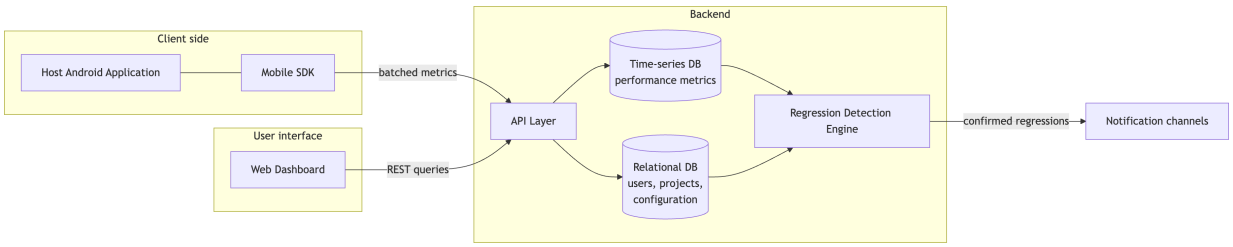

Fig. 3.1. High-Level System Architecture

3.3 Mobile SDK Design

3.3.1 Metrics Selection and Collection

Metrics selection was based on the literature review. They are divided into two major categories: continuous metrics and discrete events. Continuous metrics, such as CPU usage and frame time, are sampled at regular intervals. Discrete events, such as crashes, ANRs, and app startups, are recorded immediately when they happen. This approach allows the system to capture both the ongoing performance of the application and the rare but critical events that can significantly affect user experience.

3.3.2 UI performance metrics

For UI performance, the SDK tracks frame time, FPS, startup time, interaction delays, and ANR events.

Frame time is one of the most important metrics for measuring UI responsiveness. It represents the time needed to render a single frame of the user interface. The target frame time depends on the display refresh rate of the device. For example, on a 60 Hz display, the frame time should stay below approximately 16.67 ms in order to keep animations and interactions smooth [25].

Frame time is measured by recording the timestamps of two consecutive frames and calculating the difference between them. If t_i and t_{i-1} are the timestamps of two consecutive frames, the frame time is calculated as follows:

$$FrameTime(ms) = \frac{t_i - t_{i-1}}{10^6} \tag{3.1}$$

Frames per second (FPS) is a related metric that is derived directly from the frame time. It is calculated as:

$$FPS = \frac{1000}{FrameTime(ms)} \tag{3.2}$$

User interaction delay measures the time between a user action (e.g. tap on a screen or a swipe) and the first visible response of the application. It is obtained by recording the timestamp of the user event and the timestamp of the first frame that was rendered after the event was processed. The delay is then calculated as:

$$InteractionDelay(ms) = t_{\text{response}} - t_{\text{event}} \tag{3.3}$$

Startup time shows how long the application takes to become visually responsive after it has been launched. It is defined as the time difference between the moment the application starts and the moment when the first fully rendered frame is drawn on the screen.

ANR (Application Not Responding) events happen when the application is unable to handle user input for a long period of time. This is a critical metric, because ANRs directly affect user experience and can lead to the application being terminated by the operating system. ANR events are detected by monitoring the main thread. If the main thread remains blocked for more than a predefined threshold (for example, 5 seconds), an ANR event is recorded.

3.3.3 CPU usage

There are several ways to measure CPU usage on Android. The method selected in this work is based on the ratio between the CPU time used by the

application process and the total CPU time of the system during the same interval. The main advantage of this approach is that it does not depend on the number of CPU cores or on their frequencies, which gives a normalized measure of CPU usage. This property is important, because the collected values can be compared across different devices in a fair way. On the other hand, this method may miss short-term spikes that happen between two sampling intervals. However, this is not a problem in the context of this work, because the main goal is to detect long-term performance degradation rather than short spikes of load. It is calculated as follows:

Let ΔP be the increase in process CPU time and ΔS be the increase in total system CPU time during the same period. Then the CPU usage of the application is calculated as:

$$CPU\% = \frac{\Delta P}{\Delta S} \cdot 100 \tag{3.4}$$

3.3.4 Memory usage

There are also several methods for measuring the memory used by an Android application. In this work, memory usage is measured using the Proportional Set Size (PSS) of the application process. The PSS value represents how much physical memory is effectively used by the application, including a proportional share of the memory that is shared with other processes.

The memory usage is reported in megabytes and is calculated from the PSS value in kilobytes as:

$$MemoryUsage(MB) = \frac{PSS(KB)}{1024} \tag{3.5}$$

This metric is important, because it helps to detect memory leaks and exces-

sive memory consumption, which are frequent causes of application slowdowns and of termination by the operating system.

3.3.5 Data Aggregation and Transmission

The metrics collected on the device are not sent immediately. Instead, they are first stored in a local buffer and then transmitted in batches. Batching was chosen because opening a network connection is an energy-intensive operation, and reducing the number of network calls helps to minimize battery consumption. In addition, batching allows to reduce load on the backend.

A batch is sent when one of two conditions is satisfied: either the number of metrics in the buffer reaches a predefined value N, or the time since the last transmission exceeds a time window W.

Figure 3.2 shows the sequence of operations performed by the SDK during data collection and transmission.

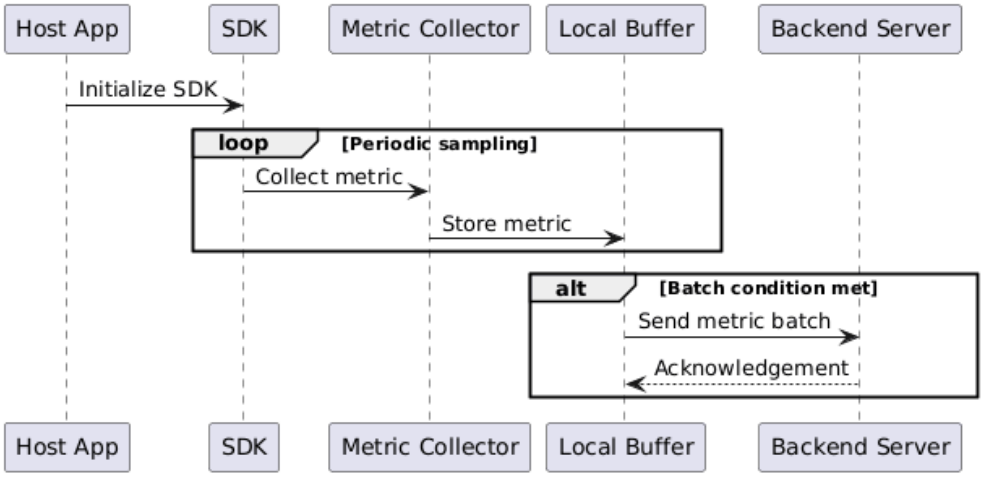

Fig. 3.2. Sequence Diagram of SDK Data Collection and Transmission

3.4 Backend Server Design

The backend has four jobs: it receives metrics from the devices, authenticates users, exposes APIs for the frontend, and runs the regression detection algorithms.

When a batch of metrics arrives from the SDK, the backend validates the request, enriches each record with device cohort information, and writes the data into the time-series database. After that, the frontend can query the stored metrics through REST endpoints, and the regression detection engine runs in the background to analyse new data as soon as it arrives.

3.4.1 Database Design

The backend uses two types of databases: a time-series database for performance metrics, and a relational database for information about users, their projects and settings. A dedicated time-series store was chosen because it is optimized to handle large volumes of time-stamped data [26].

In addition, metrics are stored only for a limited period of time. Older data is removed automatically, which helps to keep only relevant data and also reduces storage costs.

3.4.2 Performance Regression Detection Methods

Since the system works on production data without any labelled examples, the selected detection methods must not rely on supervised training. They must also be computationally efficient, easy to interpret, and suitable for continuous execution inside the backend database.

Data Partitioning and Cohorts

Before applying any detection model, the incoming data must be grouped into meaningful subsets. Comparing high-end devices directly with low-end devices could easily produce false positives, because the natural hardware limits of a low-end device might be interpreted as a software regression. To avoid this, the data is partitioned along three dimensions: the metric type (for example, CPU usage or frame time), the application screen, and the device performance cohort (Low, Medium, or High). The detection algorithms are then executed independently on each subset.

Threshold-based Methods

Threshold-based detection relies on predefined Service Level Agreements (SLAs). A cohort is flagged as abnormal if its aggregated value exceeds a fixed upper limit defined by the developer:

$$AggregatedValue > T_{\max} \tag{3.6}$$

This approach is very simple to implement and has a low computational cost. However, it requires manual configuration for each metric and each application, and the static thresholds do not adapt to natural changes in application behaviour. As a result, this method produces a high number of false positives and is not suitable for this system.

Mean Shift Detection

Mean shift detection is a statistical method that looks for regressions by comparing the mean of a current time window with the mean of a historical baseline window. A regression is reported if the current mean deviates from the baseline mean (μ_{baseline}) by more than a given multiple of the baseline standard deviation:

$$\mu_{\text{current}} - \mu_{\text{baseline}} > k \cdot \sigma_{\text{baseline}} \tag{3.7}$$

This method works well for metrics that follow a normal distribution. However, mobile performance metrics are often highly skewed and contain many outliers, which can easily distort the mean value. For this reason, the method is not very reliable for metrics such as frame rendering time.

Rolling Window Percentile Shift

Percentile-based detection focuses on the overall user experience, because it evaluates the behaviour of the worst cases rather than the average behaviour [20]. The system collects data into two windows: a baseline window (for example, the previous 7 days) and a current window (for example, the last 24 hours). A high percentile, such as the 95th percentile (P_{95}), is then calculated for both windows. A regression is reported if the relative change exceeds a predefined percentage threshold:

$$\frac{P_{95}(\text{current}) - P_{95}(\text{baseline})}{P_{95}(\text{baseline})} > \mathit{Threshold} \tag{3.8}$$

This method is robust to non-normal and skewed distributions. It is especially effective for metrics such as frame rendering time and startup time, where the tail of the distribution represents the worst user experiences.

Statistical Hypothesis Testing

To confirm that an observed performance drop is a real regression and not just random noise, a non-parametric statistical hypothesis test can be applied. In this work, the Mann-Whitney U test [27] was considered, because it compares the distributions of two samples without assuming that the data follows a normal distribution. If the resulting *p*-value is lower than the chosen significance level ($\alpha = 0.05$), the null hypothesis is rejected and the change can be considered statistically significant. This step gives high confidence in the detected regression and helps to reduce the number of false alarms.

Machine Learning and Forecasting

More advanced methods, such as ARIMA forecasting or machine learning clustering, can also be used to model complex seasonal trends in the data. However, these methods usually require a large amount of labelled data, careful parameter tuning, and significant computational resources. Considering the dynamic and bursty nature of mobile telemetry, these approaches introduce unnecessary complexity and were not selected for this lightweight system.

Selected Approach

Taking the system requirements into account, the final approach combines the **Rolling Window Percentile Shift** method with a **Statistical Hypothesis Test**, both executed on partitioned device cohorts. This hybrid approach ignores brief spikes, adapts automatically to different hardware tiers, and provides a reliable way of detecting continuous performance degradation in production environments.

3.5 Summary

This chapter presented the methodology behind the proposed performance monitoring system. The design was driven by the requirements from the literature review, with an emphasis on low overhead, energy efficiency, and robustness across devices. The mobile SDK collects metrics on the user's device and sends them to the backend in batches; the backend stores them, exposes them through an API, and runs statistical regression detection on user cohorts. Together, these parts give developers automated insight into how their applications perform in production. The next chapter moves from design to implementation and discusses the technical challenges met during the development.

Chapter 4

Implementation

This chapter presents the implementation of the design introduced in Chapter 3. Section 4.1 covers the mobile SDK, from initialization to the individual collectors and the buffering and transmission pipeline. Section 4.2 describes the backend server, focusing on user management and the ingestion path. Section 4.3 walks through the frontend web application and its main features. Section 4.4 then explains the regression detection pipeline in detail.

4.1 Mobile SDK Implementation

The mobile part of the monitoring system was implemented as an Android library written in Kotlin. Kotlin was chosen because it is the official, recommended language for Android development [28].

The SDK is distributed through a Maven repository, which allows developers to add it to their project easily by adding s dependency to their build.gradle file.

4.1.1 Initialization and Lifecycle Management

One of the requirements for the SDK was to make the integration as simple as possible for developers. To achieve this, the entire setup is done with a single call to the PerfX.initialize() method inside the application class. The developer only needs to provide the application context and a unique project ID.

Initialization runs a short sequence of steps. The AppInfoProvider collects device and app metadata (OS version, device model, app version). The SDK then opens the local Room database (MetricDatabase), starts the background sync manager, and creates the metric collectors. Finally, the SDK registers an Application.ActivityLifecycleCallbacks listener so it can follow the active screen and pause network sync while the app sits in the background, which saves battery.

Listing 4.1: SDK Initialization

```
fun initialize(application: Application, projectId: String) {
    if (isRunning) return

    appInfo = AppInfoProvider().get(application, projectId)
    db = MetricDatabase.getInstance(application)
    SyncManager.startPeriodicSync(application)

    // ... Collector initialization (described below)

    registerActivityCallback(application)
    isRunning = true
}
```

4.1.2 Performance Metrics Collectors

Each metric collector extends a common abstract class called PerformanceCollector. This class defines a single collect() method that returns a Kotlin Flow of Metric objects, which allows the SDK to treat all collectors in a uniform way.

Listing 4.2: PerformanceCollector Abstract Class

```
internal abstract class PerformanceCollector {
    abstract fun collect(intervalMs: Long): Flow<Metric>
    abstract fun stop()
}
```

All individual metric streams are merged into a single Flow inside the MetricsRepository class. This design makes it easy to add new collectors in the future without changing the rest of the pipeline.

1. Memory Collector

The RamCollector runs on a background thread and measures the memory footprint of the application at a fixed sampling interval. It uses two Android APIs: Debug.getMemoryInfo() to get the total PSS (Proportional Set Size) of the process, and the Java Runtime to calculate the current Java heap usage. The Java heap usage is calculated as the difference between the total allocated heap and the currently free heap. This value is what gets emitted as the metric, because it reflects the memory actually used by the application code.

Listing 4.3: Memory Collector Implementation

```
val memoryInfo = Debug.MemoryInfo()
Debug.getMemoryInfo(memoryInfo)

val runtime = Runtime.getRuntime()
val javaHeapUsedMb = (runtime.totalMemory() - runtime.freeMemory
```

```
    ()) / (1024.0 * 1024.0)
6 val pssMb = memoryInfo.totalPss / 1024.0
7
8 emit(Metric.MemoryUsage(timestamp = System.currentTimeMillis(),
    value = javaHeapUsedMb))
```

2. CPU Collector

Because modern Android versions restrict direct access to the /proc/stat file for security reasons, the CpuCollector uses the android.os.Process API instead. It measures CPU usage by recording the elapsed CPU time of the application process at two moments in time, separated by the sampling interval. The CPU usage percentage is then calculated as the ratio of the CPU time consumed during the interval to the total wall-clock time of that interval:

$$CPU\% = \frac{\Delta CPU_{process}}{\Delta t_{wall}} \cdot 100 \tag{4.1}$$

This approach gives a normalized measure of how much CPU time the application actually consumed relative to real time, which makes the values comparable across devices with different numbers of cores.

3. Frame Time Collector

To measure frame rendering time, the SDK registers a Choreographer.FrameCallback on the main thread. Android calls this callback every time a new frame is about to be drawn, passing the frame timestamp in nanoseconds. The collector calculates the time difference between two consecutive callbacks to get the raw frame time. Individual frame times are accumulated over a fixed interval and then averaged before being emitted as a single metric. This aggregation reduces the number of data points that need to be stored and transmitted, while still capturing the overall rendering performance of the window.

Listing 4.4: Frame Time Collector — core measurement

```
val callback = Choreographer.FrameCallback { frameTimeNs ->
    if (lastFrameTimeNs != 0L) {
        val deltaMs = (frameTimeNs - lastFrameTimeNs) / 1_000_000
            .0
        if (deltaMs >= 0.0) {
            frameCount++
            totalFrameTimeMs += deltaMs
        }
    }
    lastFrameTimeNs = frameTimeNs
    Choreographer.getInstance().postFrameCallback(frameCallback)
}

// Emitted every intervalMs
val avgFrameTime = if (frameCount > 0) totalFrameTimeMs /
   frameCount else 0.0
emit(Metric.FrameTime(timestamp = System.currentTimeMillis(),
   value = avgFrameTime))
```

4. Startup Time Collector

Startup time is measured by recording the timestamp at the moment the application process starts and comparing it to the timestamp of the first fully drawn frame. The SDK uses the reportFullyDrawn() API to detect when the first screen has finished rendering. The difference between these two timestamps is reported as the startup time.

5. Interaction Delay Collector

User interaction delay is measured by intercepting touch events on the root view of each activity. When a touch event is received, its timestamp is recorded.

The delay is then calculated as the time between the touch event and the first frame rendered after the event was processed, which is detected using the Choreographer callback.

6. ANR Detector

The SDK detects ANR events using a watchdog pattern. A background thread posts a short task to the main thread's Handler and waits for it. If the task does not run within the timeout (5 seconds by default, matching the official Android documentation [29]), the watchdog treats the main thread as blocked and records an ANR. The approach needs no special permissions and works on every Android version.

Listing 4.5: ANR Watchdog — detection logic

```
val mainHandler = Handler(Looper.getMainLooper())
var mainThreadResponded = false

mainHandler.post { mainThreadResponded = true }
Thread.sleep(ANR_TIMEOUT_MS)

if (!mainThreadResponded) {
    emit(Metric.Anr(timestamp = System.currentTimeMillis()))
}
```

4.1.3 Data Buffering and Network Transmission

The SDK uses a two-tier buffering approach to make sure that no metrics are lost if the application crashes or the network is temporarily unavailable.

Tier 1: In-Memory Buffer

As metrics arrive from the various collectors, they are first stored in a Con-

currentLinkedQueue. This data structure was chosen because multiple collectors produce data concurrently on different background threads, and using a standard list could cause race conditions. The ConcurrentLinkedQueue allows safe concurrent access without explicit locking, which keeps the overhead low.

When the number of items in the memory buffer reaches a predefined limit (MEMORY_LIMIT), the buffer is flushed to disk.

Listing 4.6: In-Memory Buffering

```
flow.collect { metric ->
    memoryBuffer.add(metric)
    memoryBufferSize += 1

    if (memoryBufferSize >= Constants.MEMORY_LIMIT) {
        flushToDisk(db, context)
        memoryBufferSize = 0
    }
}
```

Tier 2: Disk Storage and Network Synchronization

When the memory buffer is flushed, the metrics are saved to a local SQLite database managed by the Room library. Each record is saved together with the name of the currently visible screen, which is tracked by the lifecycle callback registered during initialization.

When the number of records in the database reaches DISK_LIMIT, an immediate network synchronization is triggered. In addition, a periodic background job runs every 15 minutes to synchronize any remaining records, even if the size threshold has not been reached. This ensures that metrics are uploaded even when the application is used infrequently.

The network layer is implemented using Retrofit and OkHttp. A batch

of records is retrieved from the database, serialized into JSON using Kotlinx Serialization, and sent to the backend via a single HTTP POST request. If the request succeeds, the uploaded records are deleted from the local database. If it fails, the records remain on disk and are retried during the next synchronization.

Listing 4.7: Network Synchronization

```
val batchApi = BatchApi(
    appInfo = PerfX.appInfo,
    metrics = batch.map { it.mapToApi() }
)

val response = NetworkClient.api.uploadBatch(batchApi)

if (response.isSuccessful) {
    db.metricDao().deleteMetrics(batch.map { it.id })
}
```

4.2 Backend Server Implementation

The backend server was built using Kotlin and the Ktor framework. Ktor is a lightweight, asynchronous framework that is well suited for building REST APIs. It handles each incoming request on a coroutine, which allows it to process many connections at the same time without blocking threads.

The backend uses two separate database systems: PostgreSQL for relational data such as user accounts and project settings, and ClickHouse for time-series performance metrics.

4.2.1 User and Project Management

User accounts, projects, and regression detection configurations are stored in PostgreSQL. Figure 4.1 shows the database schema.

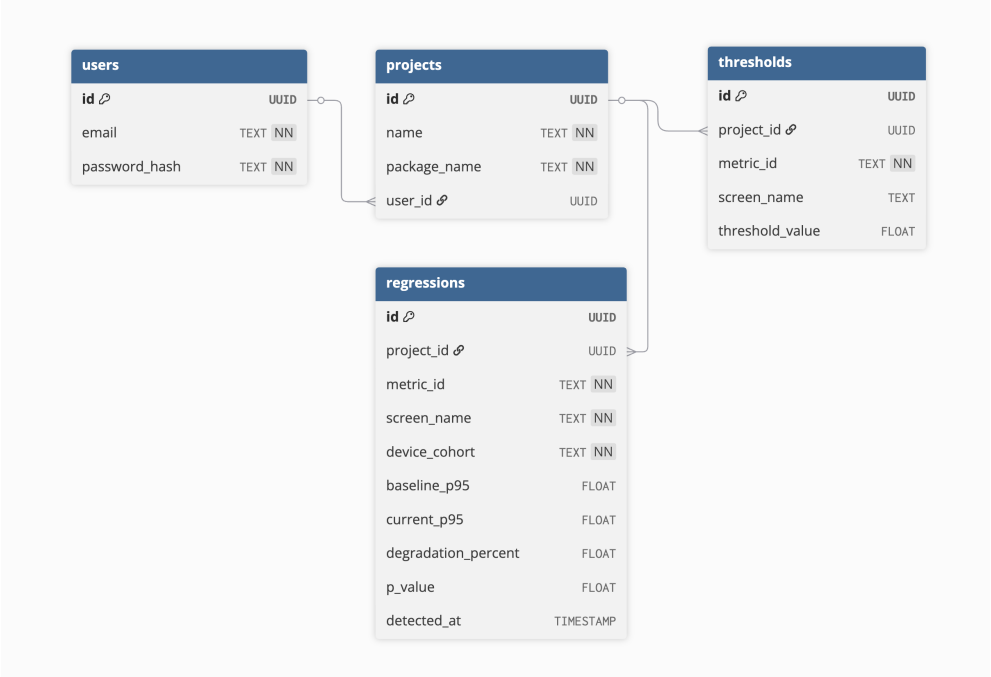

Fig. 4.1. PostgreSQL database schema

User passwords are never stored in plain text. Before saving a password, the server hashes it using the BCrypt algorithm. When a user logs in, the server verifies the provided password against the stored hash. After a successful login, the server generates a JWT token using the JwtConfig object. This token must be included in all subsequent requests to the API, which ensures that users can only access data belonging to their own projects.

When a developer creates a new project, the server checks that the project name and Android package name are not already taken. If the validation passes, the project is saved and linked to the user's account. The thresholds table allows developers to define custom P95 degradation thresholds for specific metrics and

screens. By default, the regression detection pipeline uses a global threshold of 15% for all metrics. However, different metrics have different levels of natural variability. For example, frame time is relatively stable, so even a 10% shift may indicate a real problem. CPU usage, on the other hand, fluctuates more under normal conditions, so a 15% threshold may produce too many false alarms. The thresholds table solves this by storing per-project and per-metric overrides. When the detection pipeline evaluates a cohort, it looks up the threshold using the following priority order: first, an exact match for the project, metric, and screen name; then a project-wide rule for that metric with no specific screen; and finally the global default. This design lets each developer tune the sensitivity of the system without changing any code.

4.2.2 Data Ingestion

The data ingestion endpoint is the most performance-critical part of the backend, because the mobile SDKs from many users can send large numbers of metrics at the same time. Storing this data in PostgreSQL would not be efficient enough for this workload. For this reason, all performance metrics are stored in ClickHouse, which is a column-oriented database optimized for fast batch insertions and analytical range queries over time [30].

The metric_records table was designed to support the queries needed by the regression detection pipeline efficiently. The table uses the MergeTree engine, which is ClickHouse's default storage engine optimized for fast insertions and range scans. The data is partitioned by date so that queries over a recent time window only scan the relevant partitions. The primary sort key is (project_id, metric_id, ts), which makes range queries over time for a specific project and

metric very fast.

Listing 4.8: ClickHouse metric_records table schema

```
CREATE TABLE IF NOT EXISTS metrics.metric_records
(
    project_id      String,
    package_name    String,
    ts              DateTime64(3, 'UTC'),
    metric_id       LowCardinality(String),
    screen_name     LowCardinality(String),
    value           Float64,
    app_version     LowCardinality(String),
    os_version      LowCardinality(String),
    device_model    LowCardinality(String),
    device_cohort LowCardinality(String) DEFAULT 'Unknown',
    received_at     DateTime DEFAULT now()
)
ENGINE = MergeTree
PARTITION BY toDate(ts)
ORDER BY (project_id, metric_id, ts);
```

Columns such as metric_id, screen_name, and device_cohort use the LowCardinality type, which compresses repeated string values efficiently and speeds up GROUP BY queries on these columns. Figure **??** shows the structure of the table.

When the mobile SDK sends a POST request to the /ingest endpoint, the server receives a batch of metrics together with the application and device metadata. Instead of inserting each metric individually, the ClickHouseClient formats the entire batch as a single JSONEachRow payload and executes one bulk INSERT query. Before saving each record, the server also assigns the device cohort

(Low, Medium, or High) based on the hardware metadata sent by the SDK. This batching approach keeps the load on the database low and allows the server to handle a large number of concurrent requests without becoming a bottleneck.

4.3 Frontend Implementation

The frontend was built using Streamlit, a Python framework for building interactive web applications. Streamlit was chosen because it allows data-driven dashboards to be created quickly, without the need for a separate frontend codebase. Charts and visualizations are rendered using Plotly.

4.3.1 User Authorization

Users can register and log in to the dashboard using their email and password. After a successful login, the JWT token received from the backend is stored in the browser session. All subsequent requests to the backend API include this token for authentication.

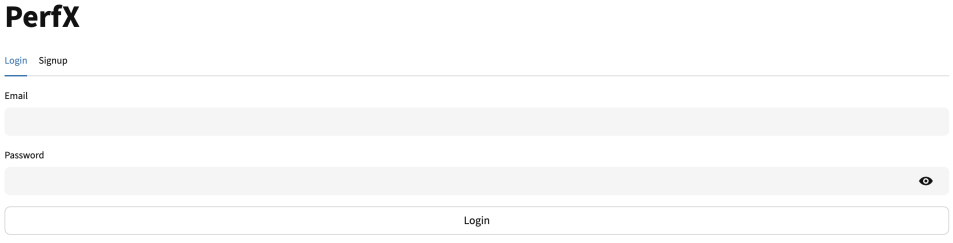

Fig. 4.2. Login page of the dashboard

4.3.2 Project Management

After logging in, users can view a list of their existing projects, create new ones, and delete old ones. When a new project is created, the dashboard displays the unique project ID that the developer needs to use when initializing the mobile SDK in their Android application.

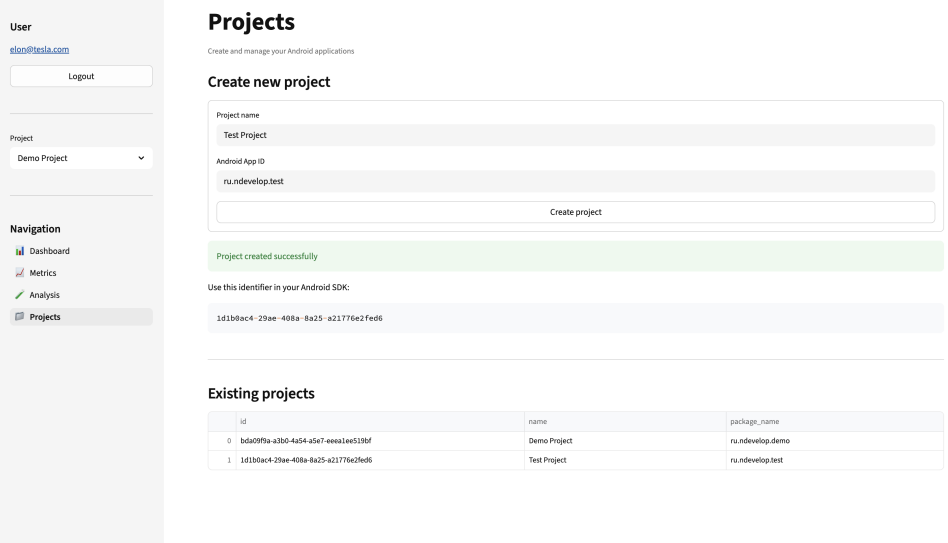

Fig. 4.3. Project list page

4.3.3 Dashboards

The dashboard page allows users to visualize the collected metrics using interactive charts. Users can filter the data by time period, screen name, and device cohort to focus on a specific part of the application or a specific group of devices.

4.3.4 Metrics Explorer

The metrics explorer page shows the raw metric data in a table. Users can browse all collected measurements along with their timestamps, values, and

associated metadata. The data can also be exported as a CSV file for further analysis in external tools such as Excel or Jupyter Notebook. This feature is useful for debugging or for performing custom analyses that go beyond what the standard dashboards offer.

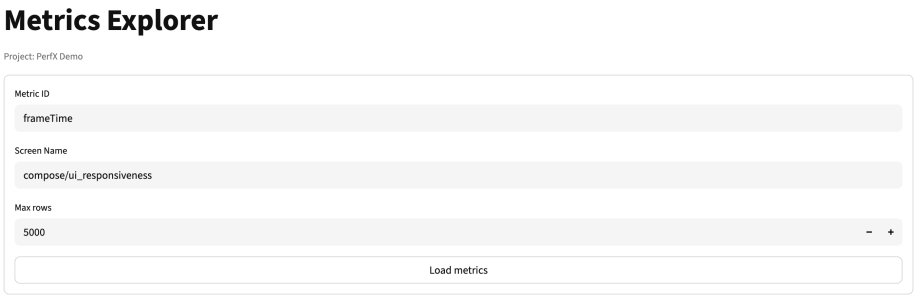

Fig. 4.4. Metrics Explorer — filter selection

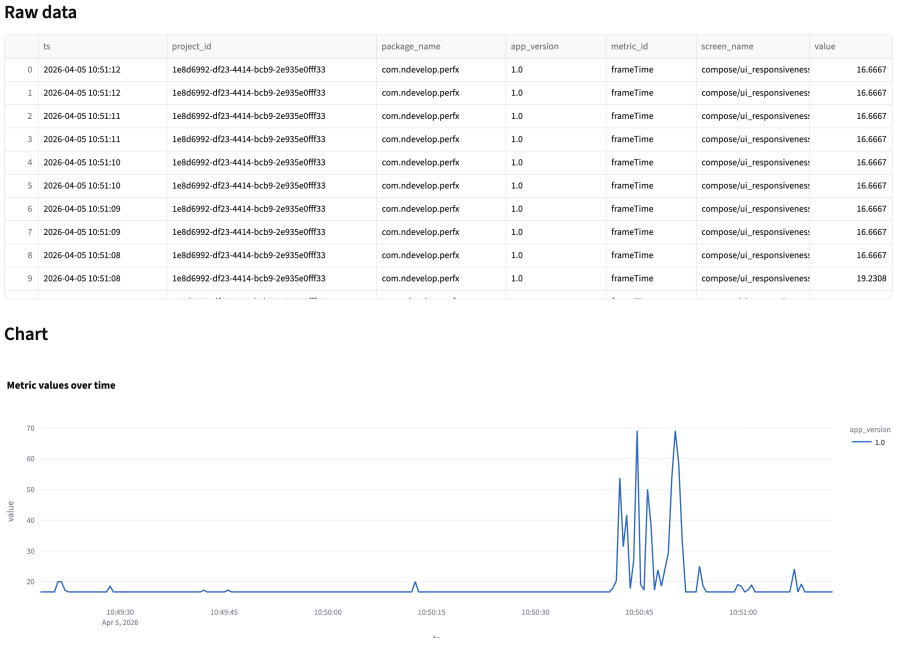

Fig. 4.5. Metrics Explorer - frame time chart with a detected regression

4.3.5 Alerting and Notifications

The alerting page allows users to configure performance thresholds for specific metrics and screens. Users can also choose how they want to be notified when a regression is detected, for example by email or via a Telegram message. The sensitivity of the regression detection algorithm can be adjusted per metric and per screen by changing the relative degradation threshold stored in the thresholds table.

4.4 Performance Regression Detection Implementation

The regression detection pipeline described in Chapter 3 was split into three stages: device cohort classification on the Ktor backend, data aggregation using ClickHouse queries, and statistical analysis using a Python service.

4.4.1 Server-Side Cohort Classification

The device cohort is calculated on the backend rather than on the mobile device. This decision was made so that the classification rules can be updated without requiring users to install a new version of the SDK. When the Ktor server receives a batch of metrics, it reads the hardware metadata (total RAM and CPU core count) from the request and assigns a cohort label before saving the record to ClickHouse.

Listing 4.9: Device cohort classification

```
1 fun calculateDeviceCohort(totalRamGb: Double, cpuCores: Int):
    String {
```

```
    return when {
        totalRamGb <= 3.0 -> "Low"
        totalRamGb <= 6.0 && cpuCores <= 8 -> "Medium"
        else -> "High"
    }
}
```

4.4.2 ClickHouse Aggregation Queries

To avoid loading large amounts of raw data into Python, the 95th percentile values for both the baseline and current windows are calculated directly in ClickHouse using the built-in quantileIf function. The query groups the results by metric type, screen name, and device cohort, which corresponds to the data partitioning described in Chapter 3.

Listing 4.10: ClickHouse query for P95 percentiles

```
SELECT
    metric_id, screen_name, device_cohort,
    quantileIf(0.95)(value, ts >= now() - INTERVAL 7 DAY
                            AND ts < now() - INTERVAL 1 DAY) AS
                                baseline_p95,
    quantileIf(0.95)(value, ts >= now() - INTERVAL 1 DAY) AS
       current_p95
FROM metric_records
GROUP BY metric_id, screen_name, device_cohort
HAVING baseline_p95 > 0
```

4.4.3 Statistical Validation

After the percentiles are retrieved from the database, the Python service calculates the relative degradation for each combination of metric, screen, and device cohort. If the degradation exceeds the threshold configured by the developer (15% by default), the system performs a Mann-Whitney U test on the raw data distributions to confirm that the change is statistically significant. The test is implemented using the scipy.stats library. If the resulting p-value is below 0.05, the regression is considered confirmed.

Listing 4.11: Mann-Whitney U test for regression confirmation

```
from scipy.stats import mannwhitneyu

def validate_regression(baseline_data, current_data,
    threshold_delta):
    delta = (current_data.p95 - baseline_data.p95) /
        baseline_data.p95

    if delta > threshold_delta:
        stat, p_value = mannwhitneyu(current_data.raw_values,
                                     baseline_data.raw_values,
                                     alternative='greater')
        if p_value < 0.05:
            return True, p_value

    return False, None
```

4.4.4 Automated Alerting

When a regression is confirmed, the Python service saves it to the regressions table in PostgreSQL, recording the metric type, affected screen, device cohort, the calculated degradation percentage, and the p-value from the statistical test. After saving, the service sends a notification to the developer through the channel they configured in the dashboard (email or Telegram). The notification includes the name of the affected screen, the device cohort, the metric type, and the size of the degradation. Confirmed regressions are also displayed on the dashboard, where developers can filter and investigate active issues.

4.5 Summary

This chapter described the implementation of all four system components. The mobile SDK was built as a Kotlin Android library that collects six performance metrics using dedicated collectors, buffers the data in a two-tier storage layer, and transmits it to the backend in batches. The backend server uses Ktor to receive the data and stores it in two separate databases: ClickHouse for time-series metrics and PostgreSQL for user and project metadata. The frontend was built with Streamlit and provides dashboards, raw data exploration, and alert configuration. Finally, the regression detection pipeline was implemented as a Python service that runs ClickHouse aggregation queries and applies a statistical hypothesis test to confirm detected regressions. The next chapter evaluates how well this system works in practice.

Chapter 5

Evaluation and Discussion

This chapter evaluates the performance monitoring system presented in Chapters 3 and 4. The evaluation is organised around three questions: how much overhead the SDK adds to a host application, how accurate its measurements are compared with established profiling tools, and how reliably the regression detection pipeline identifies performance regressions. Section 5.1 describes the validation setup. Section 5.2 reports the overhead introduced by the SDK. Section 5.3 compares the collected metrics against reference tools. Section 5.4 evaluates the regression detection pipeline on synthetic regressions. Section 5.5 interprets the results in relation to the research questions, and Section 5.6 discusses the limitations of the evaluation.

5.1 Validation Setup

The evaluation was divided into three parts: a measurement of the overhead added by the SDK (Section 5.2), a comparison of the collected metrics against reference tools (Section 5.3), and an evaluation of the regression detection pipeline

on synthetic regressions (Section 5.4).

5.1.1 Test Devices

The overhead and accuracy experiments were carried out on [TODO: number] physical Android devices. The devices were selected to cover the three performance cohorts defined in Section 3.4.2, so that the results would not be limited to a single hardware class. Table **??** lists each device together with its hardware characteristics, its Android version, and the cohort assigned by the backend.

5.1.2 Test Application

The SDK was integrated into [TODO: name and a short description of the host application used for the experiments]. [TODO: describe how the application was exercised during the experiments, for example through a fixed sequence of screens and user actions, so that each run was repeatable.]

5.1.3 Reference Tools

To check the accuracy of the collected metrics, the values reported by the SDK were compared against reference tools. Android Profiler was used as the reference for CPU usage and memory usage, because it reports per-process resource consumption during a session. System tracing provided by the Android platform was used as the reference for frame rendering time, because it records the timing of individual frames [31]. These tools are intended for use during development and are not suitable for continuous monitoring on user devices, but

under controlled conditions they are accepted as reliable references.

5.1.4 Synthetic Regression Injection

The regression detection pipeline was evaluated using synthetic regressions, that is, performance degradations introduced deliberately into the host application. Three types of synthetic regression were used: an increase in CPU load, a memory leak, and blocking of the UI thread. Each type targets a different performance metric. Increased CPU load is expected to raise the CPU usage metric, a memory leak is expected to raise the memory usage metric, and UI thread blocking is expected to raise frame time and user interaction delay.

For each experiment, the application was first run in its normal state to populate the baseline window, and the synthetic regression was then enabled to populate the current window. Because the expected effect of each injected regression is known, the output of the pipeline can be compared directly against the expected result. The exact way in which each regression was injected is described in Section 5.4.

5.2 SDK Overhead Evaluation

In the first part of evaluation, the overhead that the SDK adds to a host application was measured. To do this, the same usage scenario was run in two configurations. First, the application was run with the SDK disabled, and then with the SDK enabled. The resource usage of the application process was recorded in both configurations using the reference tools described in Section 5.1. To reduce the effect of random variation, the scenario was repeated [TODO: number] times

on each device, and the reported values are the averages of these runs. The overhead is the difference between the two configurations.

The measurements covered CPU usage and memory usage. [TODO: state whether battery consumption and network data volume were also measured, and if so, describe briefly how.] Table **??** reports the results for each test device.

Across the test devices, the SDK added on average [TODO: value]% CPU usage and [TODO: value] MB of memory. The CPU overhead of [TODO: value]% is [TODO: lower than / comparable to / higher than] the overhead of about 6% reported for the Panappticon tracing system [10]. [TODO: state whether the measured overhead meets the low-overhead requirement from Section 3.1; this claim must be supported by the values in Table **??**.]

5.3 Metric Accuracy Evaluation

The second part of the evaluation checked whether the metrics collected by the SDK are accurate.

For each metric, the value reported by the SDK was compared against the value reported by the corresponding reference tool, measured at the same time, on the same device, and under the same usage scenario. The comparison covered CPU usage, memory usage, frame time, and startup time. The deviation was calculated as the relative difference between the SDK value and the reference value. Table **??** reports the results.

The deviations were within [TODO: value]% for all four metrics. A small and consistent deviation is acceptable for this system, because the regression detection pipeline compares the relative change of a metric between the baseline and current windows rather than its absolute value (Section 3.4.2). A constant

measurement offset therefore appears in both windows and largely cancels out when the percentile shift is calculated.

5.4 Regression Detection Evaluation

The third part of the evaluation measured how well the system detects performance regressions. The synthetic regressions described in Section 5.1 were injected one at a time, and the output of the regression detection pipeline was compared against the expected result.

For these experiments, the baseline and current windows were shorter than the production values mentioned in Section 3.4.2 ([TODO: production window lengths, for example 7 days for the baseline window and 24 hours for the current window]). The windows were set to [TODO: experiment window lengths] so that a complete experiment could be carried out in a single session. The detection logic itself, including the P95 percentile shift, the degradation threshold, and the Mann-Whitney U test, was not changed.

A synthetic CPU regression was injected by [TODO: describe how CPU load was increased, for example by running an additional background computation on a specific screen]. The pipeline reported a P95 degradation of [TODO: value]% for the CPU usage metric, with a Mann-Whitney U test p-value of [TODO: value]. [TODO: state whether the regression was confirmed.]

A synthetic memory leak was injected by [TODO: describe how the leak was introduced, for example by retaining objects so that they are never released]. The pipeline reported a P95 degradation of [TODO: value]% for the memory usage metric, with a p-value of [TODO: value]. [TODO: state whether the regression was confirmed.]

A synthetic UI thread regression was injected by [TODO: describe how the UI thread was blocked, for example by performing heavy work on the main thread]. The pipeline reported a P95 degradation of [TODO: value]% for the frame time metric, with a p-value of [TODO: value]. [TODO: state whether the regression was confirmed, and whether the blocking also produced ANR events.]

Table **??** summarises the outcome of the three experiments.

In addition to the three regression experiments, a control run was carried out in which the application was used normally and no regression was injected. This run measured how many false alerts the system produces under normal conditions. In the control run, the pipeline reported [TODO: number] candidate regressions and [TODO: number] confirmed regressions. [TODO: comment on whether this matches the expected result of zero confirmed regressions.] False negatives were assessed by checking whether each injected regression was confirmed: [TODO: number] of the [TODO: number] injected regressions were detected. [TODO: if any regression was missed, state which one and explain why.]

5.5 Discussion of Results

This section interprets the results in relation to the three research questions defined in Chapter 1.

The first research question asks how a low-overhead, continuous monitoring system can be built so that it runs on real user devices without significantly affecting the user experience. The overhead measurements in Section 5.2 address this question directly. [TODO: once the overhead values are known, state whether they show that the SDK is low-overhead enough to run on real devices without a noticeable effect on the user experience.] The design choices described in

Chapter 4, in particular the two-tier buffering of metrics and the batched network transmission, were intended to keep this overhead low by reducing the number of network connections and disk writes.

The second research question asks how performance regressions can be detected accurately using statistical methods while keeping the number of false alerts low. The system addresses this with a two-stage method (Section 3.4.2). The main filter is the rolling-window P95 percentile shift: a candidate regression is reported only when the 95th percentile of a metric increases by more than the configured threshold, so small fluctuations are ignored. The Mann-Whitney U test is then applied as a secondary check that filters out changes likely to be random noise. This secondary check is most useful for screens and cohorts with little data, where a percentile shift can occur by chance; on screens with a large amount of data, the threshold does most of the filtering. The two-stage approach differs from threshold-based tools such as Firebase Performance Monitoring, which rely on fixed limits that must be tuned manually and can produce many false alerts [2]. In the control run in Section 5.4, the pipeline produced [TODO: number] confirmed false alerts. [TODO: comment on this number as evidence that the method helps to reduce false alerts; do not claim that false alerts are eliminated.] Confirming candidate regressions with a statistical test follows the same principle as large-scale systems such as FBDetect, which validates detected regressions with hypothesis testing before alerting [22].

The third research question asks how effective the system is at detecting regressions on real Android applications. The synthetic regression experiments show that the pipeline detected [TODO: number] of the [TODO: number] injected regressions. [TODO: interpret this result in one or two sentences.] However, synthetic regressions are simpler than the regressions that occur in real applications,

so this result should be read as evidence that the detection method works as designed, not as a measurement of its effectiveness in production. The limitations of this approach are discussed in Section 5.6.

5.6 Limitations

The evaluation has several limitations that should be considered when interpreting the results.

First, the regressions used in the evaluation were synthetic. They were injected in a controlled way and produce a clear and steady change in a single metric. Real performance regressions are often more gradual, may affect several metrics at the same time, and may appear only on specific devices or screens. The experiments therefore show that the detection method works under controlled conditions, but they do not measure how well it performs against real regressions.

Second, the system was not deployed to real users. The overhead and the detection quality were measured on a small set of test devices running a fixed usage scenario. Real usage is far more varied, and the behaviour of the system under that variety was not measured.

Third, the number of test devices and host applications was limited. [TODO: if the experiments covered only one device cohort or one application, state this explicitly here.] The results may not generalise to all device cohorts or to applications with a different structure or workload.

Fourth, the reference tools used in the accuracy evaluation are themselves approximations. Android Profiler and system tracing are accepted as references, but they have their own measurement error, and the deviation reported in Section 5.3 is relative to these tools rather than to an absolute ground truth.

Fifth, the statistical test used to confirm regressions has a known weakness. On screens and cohorts with a large amount of data, almost any change becomes statistically significant, so the percentile threshold does most of the filtering and the test mainly confirms regressions that the threshold has already found. The test is most useful for screens and cohorts with little data, where a percentile shift can occur by chance. The statistical analysis in this work was kept deliberately simple, and a more detailed statistical treatment is left for future work.

Finally, the baseline and current windows used in the regression experiments were shorter than the windows intended for production use. Short windows make the experiments practical, but they do not capture long-term effects such as daily or weekly usage patterns, which could affect the baseline in a real deployment.

Chapter 6

Conclusion

This thesis set out to develop a system for continuous performance monitoring and automated regression detection for Android applications. This chapter summarises the work, answers the research questions, states the main contribution, and outlines directions for future work.

6.1 Summary of the Work

This thesis has addressed the problem of detecting performance regressions in Android applications after release. Mobile applications run on a wide range of devices, and a change that keeps the application functionally correct can still make it slower or less responsive for users. Pre-release profiling tools cannot reproduce this variety, while existing post-release monitoring tools either provide no automated alerting or rely on fixed thresholds that must be tuned by hand. The thesis identified the gap between continuous post-release monitoring and automated regression detection, and set out to build a system that closes it.

To address this gap, a performance monitoring system was designed and

implemented. The system consists of four components. A mobile SDK, written in Kotlin, collects performance metrics on real user devices and sends them to a backend in batches. The backend, built with Ktor, receives the metrics, assigns each device to a performance cohort, and stores the data in ClickHouse for time-series metrics and in PostgreSQL for user and project information. A regression detection pipeline compares a recent current window against a historical baseline window, using a P95 percentile shift to find candidate regressions and a Mann-Whitney U test to confirm them. A frontend dashboard, built with Streamlit, lets developers manage projects, explore metrics, and configure alerts.

The system was validated in three steps, as described in Chapter 5. The overhead of the SDK was measured on real devices, the accuracy of the collected metrics was checked against reference tools, and the regression detection pipeline was evaluated on synthetic regressions.

6.2 Answers to the Research Questions

The thesis set out to answer three research questions.

The first research question asked how a low-overhead, continuous performance monitoring system for Android applications can be implemented so that it runs on real user devices without significantly affecting the user experience. This question was answered by the design and implementation of the mobile SDK (Chapters 3 and 4). The SDK collects CPU usage, memory usage, frame time, startup time, interaction delay, and ANR events through dedicated collectors, stores the data in a two-tier buffer, and transmits it to the backend in batches to reduce the number of network connections and disk writes. The overhead evaluation in Section 5.2 measured an average CPU overhead of [TODO: value]% and

a memory overhead of [TODO: value] MB. [TODO: state whether these results confirm that the SDK is low-overhead enough to run on real devices without significantly affecting the user experience.]

The second research question asked how performance regressions can be detected accurately using statistical methods while keeping the number of false alerts low. This question was answered by the regression detection pipeline (Section 3.4.2 and Section 4.4). The pipeline partitions the data by metric, application screen, and device cohort. For each partition, the main filter is a rolling-window P95 percentile shift, which reports a candidate regression only when the 95th percentile of a metric increases by more than the configured threshold. A Mann-Whitney U test is then applied as a secondary check that removes changes likely to be caused by random noise. In the control run reported in Section 5.4, the pipeline produced [TODO: number] confirmed false alerts. [TODO: comment on this number as evidence that the method helps to reduce false alerts; do not claim that false alerts are eliminated.]

The third research question asked how effective the proposed system is at detecting performance regressions when tested on real Android applications. This question was answered by the regression detection evaluation in Section 5.4. The pipeline was tested with three types of synthetic regression — increased CPU load, a memory leak, and UI thread blocking — and detected [TODO: number] of the [TODO: number] injected regressions. [TODO: add one sentence interpreting this result.] Because the regressions were synthetic, this result shows that the detection method works as designed under controlled conditions; its effectiveness against real regressions in production has not yet been measured.

6.3 Main Contribution

The main contribution of this thesis is a performance monitoring system that combines continuous, low-overhead post-release monitoring of Android applications with automated, server-side performance regression detection. Earlier work usually addresses only one of these two parts. Pre-release profilers provide detailed analysis under controlled conditions, post-release monitoring tools collect data from real devices but commonly provide no automated alerting or rely on fixed thresholds, and statistical regression detection methods are mostly studied for large-scale server systems rather than for mobile applications. The contribution of this work is the integration of these parts into a single system that:

- collects performance metrics from real user devices with low overhead;
- groups the collected data by metric, application screen, and device performance cohort, so that devices with different hardware are not compared directly;
- stores large volumes of time-series telemetry efficiently;
- compares recent performance against a historical baseline using a P95 percentile shift;
- confirms candidate regressions with a non-parametric statistical test before alerting;
- notifies developers automatically when a regression is confirmed.

This combination removes the need for developers to set and tune fixed thresholds by hand, and it reports regressions that are confirmed statistically

rather than triggered by a single measurement. The system is not intended to replace pre-release profiling tools, which still provide more detailed analysis during development. Instead, it complements them by covering the period after release, when the application runs on real devices under real usage conditions.

6.4 Future Work

Several directions remain for future work.

The most important next step is to evaluate the system in a real deployment. The current evaluation used synthetic regressions on a small set of test devices. Deploying the SDK in an application with real users would show how the system behaves under varied hardware, usage patterns, and network conditions, and would allow its effectiveness to be measured against real regressions rather than injected ones.

The set of collected metrics could also be extended. The current SDK collects CPU usage, memory usage, and several UI metrics. Battery consumption and network usage are also important for mobile applications [19], and adding collectors for them would give a more complete picture of application performance.

The regression detection method could be made more adaptive. At present, the relative degradation threshold is configured per metric and per screen. Future work could replace this fixed threshold with a method that derives a suitable threshold from the historical data of each cohort, which would further reduce the need for manual configuration. The detection pipeline could also be extended to account for recurring daily or weekly usage patterns, which the current fixed-window comparison does not capture.

The device cohort classification could be refined. Cohorts are currently

assigned from total RAM and CPU core count. Additional hardware and software characteristics, such as the CPU model or the Android version, could produce cohorts that group devices with similar real-world performance more accurately.

Finally, the system could be extended to help developers find the cause of a regression. The backend already stores the application version with each metric. Future work could link a confirmed regression to the application version in which it first appeared, which would help developers connect a regression to a specific release.

Bibliography cited

[1] Y. Tang, H. Wang, X. Zhan, *et al.*, "A systematical study on application performance management libraries for apps," *IEEE Trans. Softw. Eng.*, vol. 48, no. 8, pp. 3044–3065, Aug. 2022, ISSN: 0098-5589. DOI: 10.1109/TSE.2021.3077654.

[2] Firebase. "Firebase performance monitoring." Accessed: 30 Oct 2025. (2025), [Online]. Available: https://firebase.google.com/docs/perf-mon.

[3] Z. Dong, Y. Zhao, T. Liu, *et al.*, "Same app, different behaviors: Uncovering device-specific behaviors in android apps," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '24, Sacramento, CA, USA: Association for Computing Machinery, 2024, pp. 2099–2109, ISBN: 9798400712487. DOI: 10.1145/3691620.3695272.

[4] M. Kamran, J. Rashid, and M. Nisar, "Android fragmentation classification, causes, problems and solutions," *International Journal of Computer Network and Information Security*, vol. 14, pp. 992–999, Sep. 2016.

[5] X. Wei, L. Gomez, I. Neamtiu, and M. Faloutsos, "Profiledroid: Multi-layer profiling of android applications," Aug. 2012. DOI: 10.1145/2348543.2348563.

[6] S. Jeong, K. Lee, J.-Y. Hwang, S. Lee, and Y. Won, "Androstep: Android storage performance analysis tool," in *Software Engineering*, 2013.

[7] F. Bouaffar, O. L. Goaer, and A. Noureddine, "Powdroid: Energy profiling of android applications," in *2021 36th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW)*, 2021, pp. 251–254. DOI: 10.1109/ASEW52652.2021.00055.

[8] L. R. Sivalingam, J. Padhye, S. Agarwal, R. Mahajan, I. Obermiller, and S. Shayandeh, "Appinsight: Mobile app performance monitoring in the wild," Nov. 2012.

[9] Y. Luo, K. Rodrigues, C. Li, *et al.*, "Hubble: Performance debugging with In-Production, Just-In-Time method tracing on android," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, Carlsbad, CA: USENIX Association, Jul. 2022, pp. 787–803, ISBN: 978-1-939133-28-1.

[10] L. Zhang, D. Bild, R. Dick, Z. Mao, and P. Dinda, "Panappticon: Event-based tracing to measure mobile application and platform performance," Sep. 2013, pp. 1–10. DOI: 10.1109/CODES-ISSS.2013.6659020.

[11] D. K. Hong, A. Nikravesh, Z. M. Mao, M. Ketkar, and M. Kishinevsky, "Perfprobe: A systematic, cross-layer performance diagnosis framework for mobile platforms," in *2019 IEEE/ACM 6th International Conference on Mobile Software Engineering and Systems (MOBILESoft)*, 2019, pp. 50–61. DOI: 10.1109/MOBILESoft.2019.00018.

[12] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Comput. Surv.*, vol. 41, no. 3, Jul. 2009, ISSN: 0360-0300. DOI: 10.1145/1541880.1541882.

[13] S. Schmidl, P. Wenig, and T. Papenbrock, "Anomaly detection in time series: A comprehensive evaluation," *Proc. VLDB Endow.*, vol. 15, no. 9, pp. 1779–1797, May 2022, ISSN: 2150-8097. DOI: 10.14778/3538598.3538602.

[14] O. Ibidunmoye, F. Hernandez-Rodriguez, and E. Elmroth, "Performance anomaly detection and bottleneck identification," *ACM Computing Surveys*, vol. 48, Jun. 2015. DOI: 10.1145/2791120.

[15] W. Liu, F. Lin, L. Guo, T.-H. Chen, and A. Hassan, "Guiwatcher: Automatically detecting gui lags by analyzing mobile application screencasts," Feb. 2025. DOI: 10.48550/arXiv.2502.04202.

[16] L. Zhang, M. Gordon, R. Dick, Z. Mao, P. Dinda, and L. Yang, "Adel: An automatic detector of energy leaks for smartphone applications," Oct. 2012, pp. 363–372. DOI: 10.1145/2380445.2380503.

[17] J. Fu, Y. Wang, Y. Zhou, and X. Wang, "How resource utilization influences ui responsiveness of android software," *Information and Software Technology*, vol. 141, p. 106 728, Sep. 2021. DOI: 10.1016/j.infsof.2021.106728.

[18] B. Li, Q. Zhao, S. Jiao, and X. Liu, "Droidperf: Profiling memory objects on android devices," Jul. 2023, pp. 1–15. DOI: 10.1145/3570361.3592503.

[19] R. Rua and J. Saraiva, "A large-scale empirical study on mobile performance: Energy, run-time and memory," *Empirical Softw. Engg.*, vol. 29, no. 1, Dec. 2023, ISSN: 1382-3256. DOI: 10.1007/s10664-023-10391-y.

[20] T. H. D. Nguyen, B. Adams, Z. M. Jiang, A. E. Hassan, M. Nasser, and P. Flora, "Automated detection of performance regressions using statistical process control techniques," in *Proceedings of the 3rd ACM/SPEC International Conference on Performance Engineering (ICPE '12)*, ACM, 2012, pp. 299–310.

[21] W. Shang, A. E. Hassan, M. Nasser, and P. Flora, "Automated detection of performance regressions using regression models on clustered performance counters," in *Proceedings of the 6th ACM/SPEC International Conference on Performance Engineering*, ACM, 2015, pp. 15–26.

[22] D. Y. Yoon, Y. Wang, M. Yu, *et al.*, "FBDetect: Catching tiny performance regressions at hyperscale through in-production monitoring," in *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles (SOSP '24)*, ACM, 2024.

[23] A. Oliner, A. Iyer, I. Stoica, E. Lagerspetz, and S. Tarkoma, "Carat: Collaborative energy diagnosis for mobile devices," Nov. 2013. DOI: 10.1145/2517351.2517354.

[24] A.-D. Schmidt, F. Peters, F. Lamour, S. Albayrak, and S. Camtepe, "Monitoring smartphones for anomaly detection," vol. 14, Feb. 2008, p. 40. DOI: 10.1007/s11036-008-0113-x.

[25] Android Developers. "Slow rendering." Accessed: 19 April 2026. (2026), [Online]. Available: https://developer.android.com/topic/performance/vitals/render.

[26] S. K. Jensen, T. B. Pedersen, and C. Thomsen, "Time series management systems: A survey," *IEEE Transactions on Knowledge and Data Engineer-*

ing, vol. 29, no. 11, pp. 2581–2600, 2017. DOI: 10.1109/TKDE.2017.2740932.

[27] H. B. Mann and D. R. Whitney, "On a test of whether one of two random variables is stochastically larger than the other," *The Annals of Mathematical Statistics*, vol. 18, no. 1, pp. 50–60, 1947. DOI: 10.1214/aoms/1177730491.

[28] Android Developers. "Android's kotlin-first approach." Accessed: 19 April 2026. (2026), [Online]. Available: https://developer.android.com/kotlin/first.

[29] Google Android. "Android vitals: App quality." Accessed: 30 Oct 2025. (2025), [Online]. Available: https://developer.android.com/topic/performance/vitals.

[30] R. Schulze, T. Schreiber, I. Yatsishin, R. Dahimene, and A. Milovidov, "ClickHouse - lightning fast analytics for everyone," *Proceedings of the VLDB Endowment*, vol. 17, no. 12, pp. 3731–3744, 2024. DOI: 10.14778/3685800.3685802.

[31] Android Developers. "Overview of system tracing." Accessed: 20 Oct 2025. (2025), [Online]. Available: https://developer.android.com/topic/performance/tracing.